

Session 13: Detailed Explanation and Examples

1. Introduction to STLC and V Model

STLC (Software Testing Life Cycle)

The **Software Testing Life Cycle (STLC)** refers to the structured approach and stages followed during the software testing process. It defines the various phases involved in testing to ensure the quality of software applications. STLC involves both **manual** and **automated** testing.

Phases of STLC:

1. **Requirement Analysis:** In this phase, the testing team analyzes the requirement documents to understand the testing scope and create test plans. Testers identify any ambiguous requirements and seek clarifications.
 - **Example:** In an e-commerce application, testers might review requirements for the checkout process to identify any test scenarios related to payment methods.
2. **Test Planning:** The team prepares the test strategy, outlines testing objectives, selects testing tools, defines the test environment, and identifies resources.
 - **Example:** Test plans are made to decide which features to test first and what resources are required, like hardware or software configurations.
3. **Test Design:** This phase involves creating detailed test cases, test scripts, and defining test data. The test cases describe the specific scenarios to test the application.
 - **Example:** Creating test cases for different shopping cart scenarios, like adding items, applying discounts, or checking the final amount.
4. **Test Environment Setup:** The test environment is prepared, which includes setting up hardware, software, network configurations, and installing the required tools.
 - **Example:** Setting up a testing server with the required database for the e-commerce website.
5. **Test Execution:** This is when the tests are actually executed. Testers run manual or automated tests and log any defects found.
 - **Example:** Running test cases to verify if the cart functionality is working as expected, logging any issues like incorrect price calculations.
6. **Defect Reporting and Tracking:** Any defects identified during the testing process are reported to developers. Testers track and verify defect fixes.
 - **Example:** Reporting an issue where the website fails to apply a discount during checkout.
7. **Test Closure:** This phase involves closing the testing process after all test cases have been executed. Test results are documented, and testing is formally concluded.
 - **Example:** Preparing a test summary report that includes test coverage, the number of test cases executed, defects found, and defects resolved.

V Model (Verification and Validation Model)

The **V-Model** is a software development model that emphasizes validation and verification at each stage of the development cycle. It's closely tied to the **Waterfall Model**, with a focus on corresponding testing phases during development.

Phases of the V Model:

- **Requirement Analysis** corresponds to **Acceptance Testing**.
- **System Design** corresponds to **System Testing**.
- **High-Level Design** corresponds to **Integration Testing**.
- **Low-Level Design** corresponds to **Unit Testing**.
- **Coding** corresponds to **Implementation Testing**.

The V-Model ensures that testing is conducted as each part of the software is being built, making the process more efficient and reducing defects.

Example: If the software is being developed for a banking application, the **High-Level Design** phase (involving features like account creation) would directly correspond to **Integration Testing**, ensuring that all components integrate smoothly.

2. Types of Testing: Manual and Automation

Manual Testing

Manual Testing is the process of manually checking software for defects. Testers execute test cases without using any automation tools. This approach requires testers to perform tests, report issues, and verify software behavior.

Advantages of Manual Testing:

- Great for **exploratory testing** where testers are not following predefined scripts.
- More flexible and useful for **usability** or **ad-hoc testing**.
- Best for **small projects** or projects that don't require repetitive tasks.

Example: A tester might manually test a login page by entering different types of usernames and passwords to check for functionality, error messages, and user experience.

Automation Testing

Automation Testing uses scripts and software tools to automatically execute predefined test cases. It is typically used to test large applications, regression tests, and repetitive test cases. Automated tests can be run frequently and consistently.

Advantages of Automation Testing:

- Saves time for **repetitive tasks**.
- Increases **test coverage** by running multiple tests simultaneously.
- Tests can be repeated multiple times with the same conditions, ensuring reliability.

Example: For the same login page, an automation script could be written to input usernames and passwords, submit the form, and check for expected results without human intervention.

3. Tools Used for Automation Testing

There are several popular tools available for automating software testing. Some widely used automation testing tools include:

- **Selenium:** Selenium is one of the most popular open-source tools for automating web applications. It supports multiple programming languages such as Java, Python, and C#. It is primarily used for functional testing of web applications.
 - **Example:** Selenium WebDriver can be used to automate browser actions like clicking buttons and entering data into forms.
 - **JUnit/TestNG:** These are widely used frameworks for **unit testing** in Java applications. They provide annotations to define test cases and manage execution.
 - **Example:** Using TestNG, you can create test methods that check various features of the application, like login and registration.
 - **Jenkins:** Jenkins is an open-source automation server often used for continuous integration (CI) and continuous testing.
 - **Example:** Jenkins can automatically run your test suite each time new code is pushed to a version control system, ensuring that defects are caught early.
 - **Appium:** Appium is an open-source tool used for automating mobile application testing (both Android and iOS). It supports a variety of programming languages.
 - **Example:** Appium can be used to automate the process of clicking on app buttons and verifying screen transitions for a mobile app.
 - **Cucumber:** Cucumber is a tool used for **Behavior-Driven Development (BDD)** that allows users to write test scenarios in natural language.
 - **Example:** Using Cucumber, testers and developers can write test scenarios in simple language like "Given the user is logged in, when they click on the cart, then the checkout page is displayed."
-

4. Introduction to Testing Methods: White-box, Black-box, and Grey-box

White-box Testing

White-box testing (also known as clear-box or glass-box testing) involves testing the internal structures or workings of an application. The tester has access to the source code, algorithms, and architecture of the system.

- **Focus:** Internal logic, data flow, control flow, and code paths.
- **Examples of White-box Testing Techniques:**
 - **Unit Testing:** Testing individual functions or methods.
 - **Code Coverage Analysis:** Ensuring that all code paths are executed.

Example: Testing the functionality of a function that adds two numbers by checking if all branches of the code are covered, such as checking for negative numbers or zero inputs.

Black-box Testing

Black-box testing involves testing the system based on its functionality, without knowledge of the internal code. Testers focus on inputs and outputs, ensuring that the system behaves as expected from the user's perspective.

- **Focus:** Behavior of the system, functionality, and user experience.
- **Examples of Black-box Testing Techniques:**
 - **Functional Testing:** Verifying features work as specified.
 - **Regression Testing:** Ensuring new code changes don't break existing features.

Example: In testing a login form, black-box testing would involve entering various combinations of usernames and passwords and verifying whether the system responds correctly without knowledge of the underlying code.

Grey-box Testing

Grey-box testing is a combination of both **white-box** and **black-box** testing. Testers have partial knowledge of the internal workings of the system, and they use that knowledge to design better test cases.

- **Focus:** Functionality testing with partial knowledge of the internal system.
- **Examples of Grey-box Testing Techniques:**
 - **Integration Testing:** Testing the interaction between modules or components.
 - **Security Testing:** Evaluating system vulnerabilities with some knowledge of the architecture.

Example: A tester might know the database structure and write queries to verify that a login attempt updates the user status correctly in the database, even though they don't have access to the actual source code.

5. Introduction to Functional Testing

Functional Testing refers to testing the functionality of an application to ensure it works as expected according to the requirements. It focuses on testing specific actions and features of the software, like user interactions, form submissions, or database operations.

Types of Functional Testing:

- **Unit Testing:** Testing individual units or components of a software (often done by developers).
- **Integration Testing:** Testing the interactions between different software modules.
- **System Testing:** Testing the entire application in an environment that simulates real user conditions.
- **User Acceptance Testing (UAT):** Testing done by end-users to verify if the software meets their needs.

Example: Testing the "Add to Cart" feature in an e-commerce site, ensuring that the product appears in the cart and the price is correct.

6. Introduction to Non-Functional Testing

Non-Functional Testing focuses on testing aspects of the software that are not related to specific behaviors or functionalities. It ensures that the system meets various performance, security, and usability standards.

Types of Non-Functional Testing:

- **Performance Testing:** Determines how well the system performs under various loads. Includes **Load Testing**, **Stress Testing**, and **Scalability Testing**.
- **Security Testing:** Tests the software for vulnerabilities and security loopholes.
- **

Usability Testing:** Evaluates how easy and user-friendly the system is for end-users.

- **Compatibility Testing:** Ensures the application works on different browsers, devices, or operating systems.

Example: Performance testing on a website to ensure it can handle a high number of simultaneous users without crashing, or security testing to check if user data is encrypted properly.

Summary:

- **STLC** outlines the stages of the software testing process, ensuring thorough testing from planning to closure.
- The **V-Model** emphasizes parallel development and testing activities, ensuring early defect detection.
- **Manual testing** is done by human testers, while **automation testing** uses scripts to execute test cases.
- Testing methods like **white-box**, **black-box**, and **grey-box** cater to different testing needs, from detailed internal checks to high-level functionality verification.
- **Functional testing** focuses on features and actions, while **non-functional testing** addresses performance, security, and usability issues.

By understanding these concepts, you can implement a comprehensive and effective testing strategy that ensures both functional and non-functional quality for your software.